

PART III. DATA / DATABASE DESIGN

15 points

SALINLAHI uses a relational database for session results and local key-value storage for player identity and progress. The split is deliberate: evaluation data must be collected centrally, while progress must work offline.

IMPLEMENTED NOW - SUPABASE (POSTGRESQL)

player_scores		
player_id	uuid	PK
player_name	text	not null
score	integer	not null
attempts	integer	not null
level	text	not null
remarks	text	
created_at	timestampz	default now()

Current design is a single denormalised session table: every row carries the nickname, so no join is needed to show recent scores.

IMPLEMENTED NOW - LOCAL (PLAYERPREFS)

PlayerPrefs (device key-value store)	
SALINLAHI_Nickname	string
SALINLAHI_HighestUnlockedLevel	int

Written by NicknameManager and LevelProgress; survives app restarts.

PLANNED - NORMALISATION FOR PER-PLAYER HISTORY

1 : N

players (not yet implemented)		
player_uid	uuid	PK
nickname	text	unique
created_at	timestampz	

player_scores.player_uid would become a FK to players.player_uid (1 : many).

Figure 2. Current data design, with the planned normalisation shown dashed.

Relational schema — Supabase (PostgreSQL)

Table: player_scores. One row is written per completed level. There is exactly one table at present; no foreign keys exist yet, which is why no join is required to display recent results.

Field	Data type	Key / constraint	Purpose
player_id	uuid	Primary key, gen_random_uuid()	Uniquely identifies the saved session record
player_name	text	Not null	Nickname of the child who played the session
score	integer	Not null, default 0	Number of Filipino words pronounced correctly
attempts	integer	Not null, default 0	Total voice attempts made, correct and incorrect
level	text	Not null	Which level the record belongs to
remarks	text	Nullable	Readable summary, for example 3/5 words correct
created_at	timestampz	Not null, default now()	When the record was saved; used to order recent results

Local storage — Unity PlayerPrefs

Storage key	Data type	Identifier	Purpose
SALINLAHI_Nickname	string	Fixed key	The current player's nickname, read by any scene and written into each saved record
SALINLAHI_HighestUnlockedLevel	int	Fixed key	Highest level reached, used by the map to lock and unlock the four environments

PlayerPrefs is Unity's built-in per-device key-value store. It is chosen over a file or a database call because both values are tiny, needed at startup, and must not depend on connectivity.

Data integrity and validation rules

Rule	Where it is enforced
player_id must be unique	Database — primary key with a generated UUID default, so the client never supplies it
player_name and level must not be empty	Database not null constraints; the client also substitutes a default nickname rather than sending a blank
Score and attempts cannot be negative	Client — both are incremented only, never decremented. A database check constraint is planned
Score must never exceed attempts	Client — a correct answer increments both counters in the same call
One record per completed playthrough	Client — the tracker refuses a second save for the same run, and resets the totals after a successful save
Records may be added and read, never changed or deleted	Database — Row Level Security grants the anon role insert and select only; no update or delete policy exists
Credentials must not enter the repository	Repository — the config asset is git-ignored and only the anon public key is ever used